

CS F407: Neural Models

AI Laboratory

Tasks 1–2: Problem and model

$\mathcal{X} = \{0, 1\}^2$, $\mathcal{Y} = \{0, 1\}$. Examples are $(0, 0) \mapsto 0$, $(0, 1) \mapsto 1$, $(1, 0) \mapsto 1$, $(1, 1) \mapsto 0$. The positive and negative line segments share midpoint $(1/2, 1/2)$, so a straight decision boundary cannot separate the classes. A single affine sigmoid classifier cannot represent XOR. Its measured final mean BCE is 0.693147, with probabilities close to 0.5.

The baseline is $2 \rightarrow 2 \rightarrow 1$, with tanh hidden activation, one output logit, and `BCEWithLogitsLoss`. Sigmoid gives the binary probability. Nonlinearity is necessary because composing affine maps still gives an affine map. Success requires lower loss, four correct labels, and a checked gradient. Repeated seeds examine initialisation sensitivity.

Tasks 3–4: Training and diagnostics

All activation runs use seed 0, float64 CPU tensors, Adam, learning rate 0.03, and 5000 full-batch steps. Only the hidden activation changes. The first tanh attempt used seed 7 and gave loss 0.346587 and labels $(0, 1, 0, 1)$. Changing only the seed to 0 gave the successful baseline; `tuning.json` records both attempts.

Activation	Initial loss	Final loss	4/4 correct	Early $\ \nabla W^{(1)}\ _2$
Sigmoid	0.707163	0.000174286	Yes	0.009625
Tanh	0.710176	6.305e-05	Yes	0.043654
Relu	0.712050	0.693147	No	0.041242

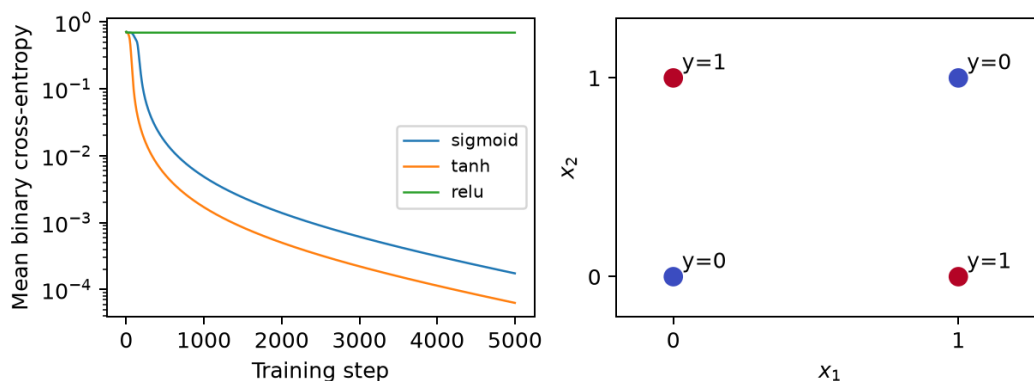
Input	Baseline probability	Label
(0, 0)	0.00004709	0
(0, 1)	0.99990929	1
(1, 0)	0.99992830	1
(1, 1)	0.00004269	0

After the first backward pass,

$$\frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} 0.01817984 & 0.03576583 \\ 0.01696792 & 0.00284001 \end{bmatrix}.$$

`model(X)` performs the forward pass, the criterion forms mean loss, `backward()` accumulates derivatives, and `optimiser.step()` updates parameters. The mean loss gradient is the mean of four example gradients. Direct averaging differs by at most 3.5×10^{-18} . Central differences with step 10^{-5} differ by at most 6.7×10^{-12} .

Sigmoid and tanh learn XOR here; ReLU does not. Every final ReLU preactivation is positive, making its hidden map affine on the four inputs. This is not a universal ranking. Inspecting preactivations also distinguishes sigmoid saturation at large $|z|$ from inactive ReLU at negative z ; neither explains this all-active ReLU result.



Symmetry. Zeroing weights retains random hidden biases (0.205095, 0.411745). Weight rows start equal but later diverge:

Before update	$\ W_1^{(1)} - W_2^{(1)}\ _2$
0	0.000000
1	0.000000
10	0.008176
100	6.650140
1000	11.244469
4999	13.195403

This run learns all labels with loss 4.64343e-05. Unequal biases prevent full unit symmetry. An additional control with all weights and biases zero keeps the hidden rows identical through 100 updates and leaves loss at log 2. Identical units receive identical gradients and cannot learn distinct features.

Tanh seed	Final loss	4/4 correct
0	6.305e-05	Yes
1	0.346589	No
2	5.60219e-05	Yes

Seed 1 stalls while seeds 0 and 2 succeed. Representation capacity does not guarantee convergence from every initialisation.

Task 5: Three-class extension

The tanh hidden layer remains $2 \rightarrow 2$. The output is $2 \rightarrow 3$, with weight shape (3, 2) and labels (0, 1, 1, 2). Softmax gives $p_k = e^{z_k} / \sum_j e^{z_j}$, so its probabilities sum to one. For cross-entropy, each example has logit gradient $p - y$; its contribution to the four-example mean is $(p - y)/4$.

Input	p_0	p_1	p_2	Class
(0, 0)	0.99998232	0.00001729	0.00000039	0
(0, 1)	0.00000899	0.99998363	0.00000739	1
(1, 0)	0.00000759	0.99998535	0.00000706	1
(1, 1)	0.00000007	0.00002191	0.99997802	2

Final loss is 1.76715e-05. Maximum row-sum error is 1.1×10^{-16} ; adding 100 to every logit changes probabilities by at most 1.6×10^{-19} .

Subtracting the maximum logit preserves the distribution while preventing overflow in exponentiation.

Reflection answers

1. Affine depth does not separate XOR. Hidden nonlinearity permits the representation learned by the successful baseline.
2. Lower loss and four correct labels show a useful learning signal, supported by independent gradient checks.
3. Fully identical weights and biases preserve symmetry. The weights-only run shows that the bias condition matters.
4. Activations change derivatives and the realised map; the table reports outcomes for one controlled initialisation.
5. Binary BCE and multiclass cross-entropy encode different targets, so output dimensions and loss must match the task.
6. The LLM supplied training and diagnostics. Tensor shapes, gradients, and initialisation assumptions still required verification.
7. At scale retain loss, label, finite-value, and probability checks. Exhaustive finite differences become expensive; sample parameters.

Large-vocabulary next-token prediction uses the same softmax, cross-entropy, and $p - y$ derivative. Embeddings, attention, context length, and computation grow substantially.

Reproduce. Run `../venv/bin/python solution.py` here. Results: `results.json`; prompt and correction record: `prompts.txt`.